

1. Explain hash file organization technique with example and its advantages and disadvantages? ***

Hash File Organization

Hash File Organization is a file organization technique in which a **hash function** is used to directly determine the address (bucket) of a record **without using an index structure**.

Easy Meaning

Instead of searching through an index, the system uses a **hash function** to directly find the bucket where the record is stored.

Working of Hash File Organization

- Let **K** be the set of all **search-key values**.
- Let **B** be the set of all **bucket addresses**.
- A **Hash Function $h(K)$** maps a search key to a bucket address.
- During insertion, the hash function calculates the bucket number and stores the record in that bucket.

Example

Suppose there are **10 buckets (0–9)** and we use a hash function to calculate the bucket number.

$h(\text{Perryridge}) = 5$

So, **Perryridge** is stored in **Bucket 5**.

$h(\text{Round Hill}) = 3$

So, **Round Hill** is stored in **Bucket 3**.

$h(\text{Brighton}) = 3$

So, **Brighton** is also stored in **Bucket 3**.

This shows that **more than one record can be mapped to the same bucket**.

Advantages

1. **No Index Required**
 - No separate index structure is needed.
2. **Fast Data Access**

- The hash function directly finds the bucket, making searching very fast.
3. **Efficient Retrieval**
- Records can be retrieved quickly using the search key.

Disadvantages

1. Bucket Overflow

A bucket may become full when many records are mapped to the same bucket.

2. Insufficient Buckets

If the number of buckets is too small compared to the number of records, overflow occurs.

3. Bucket Skew

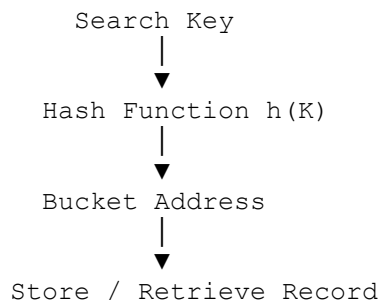
Some buckets may contain many records while others remain almost empty because of:

- Duplicate search keys.
- Poor hash function (non-uniform distribution).

4. Lookup Overhead

If multiple records are stored in the same bucket, the system must search through them one by one, which increases search time.

Simple Diagram



2. Why bucket overflow. Handing of bucket overflows? (Slide-65)

Why Bucket Overflow Occurs

□ In case of insertion, if the bucket does not have enough space, a bucket overflow is said to occur. Bucket overflow can occur mainly for two reasons:

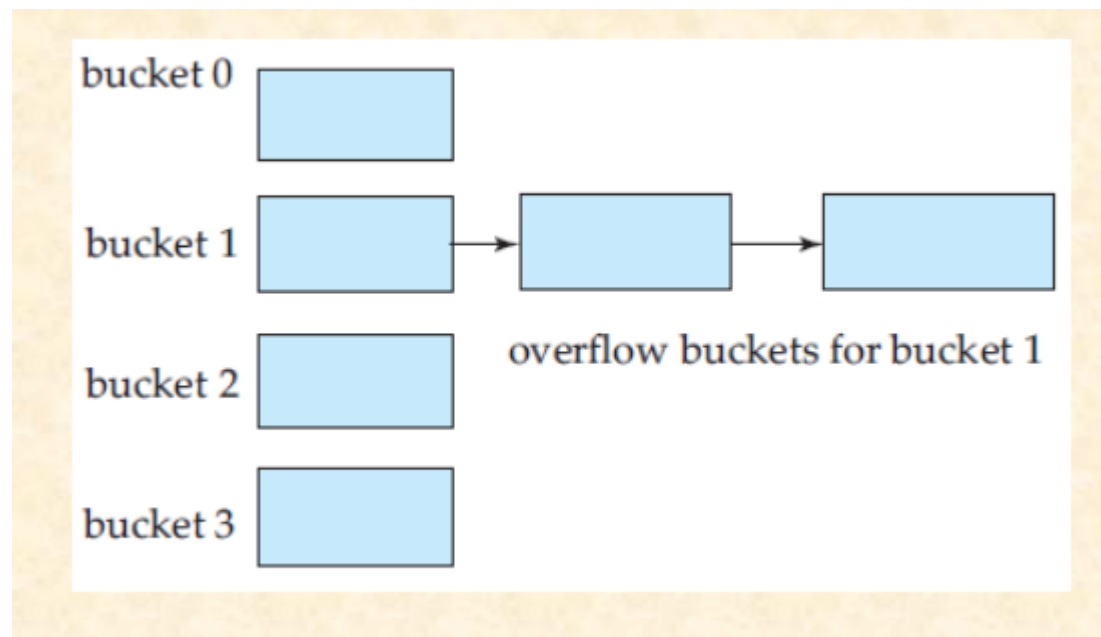
□ **Insufficient buckets:** The number of buckets n_B must be chosen such that $n_B > n_r / f_r$, where n_r denotes the total number of records that will be stored and f_r denotes the number of records that will fit in a bucket.

□ **Skew:** Some buckets are assigned more records than are others, so a bucket may overflow even when other buckets still have space. This situation is called bucket skew. Skew can occur for two reasons:

- Multiple records may have the same search key.
- The chosen hash function may result in non-uniform distribution of search keys.

Handling of Bucket Overflows (The Solution)

If a record must be inserted into a bucket b , and b is already full, the system provides an overflow bucket for b , and inserts the record into the overflow bucket. If the overflow bucket is also full, the system provides another overflow bucket, and so on. All the overflow buckets of a given bucket are chained together in a linked list.



3. Why bucket overflow. Handling of bucket overflows. Explain with example. (Slide-65) ***

2 No Question ans and see here only example

Example

Suppose there are **4 buckets (Bucket 0, Bucket 1, Bucket 2, Bucket 3)**, and each bucket can store only **2 records**.

The hash function is:

$$h(\text{key}) = \text{key} \% 4$$

Insert the following keys: **5, 9, 13, 17, 21**

Key Calculation Bucket

5 $5 \% 4 = 1$ Bucket 1

9 $9 \% 4 = 1$ Bucket 1

13 $13 \% 4 = 1$ Bucket 1

17 $17 \% 4 = 1$ Bucket 1

21 $21 \% 4 = 1$ Bucket 1

Bucket 0 : []

Bucket 1 : [5, 9] ← Full

|

Overflow 1 : [13, 17] ← Full

|

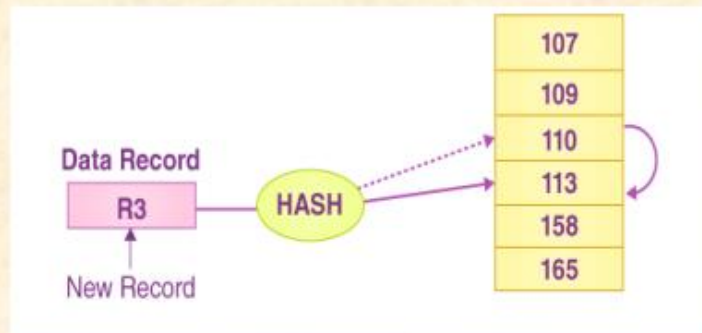
Overflow 2 : [21]

Bucket 2 : []

Bucket 3 : []

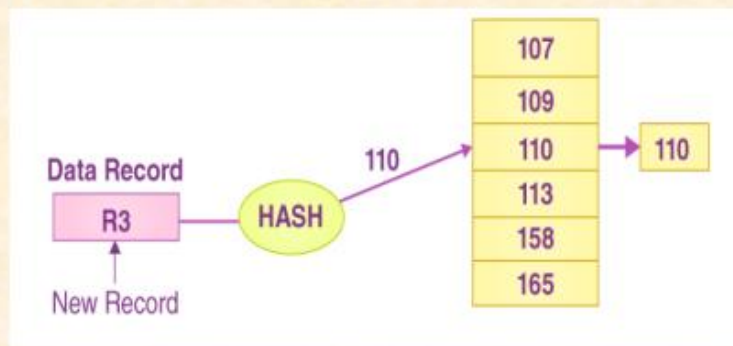
❑ Open Hashing

- Whenever a hash function generates any address that already contains data, the next bucket is assigned to it, and this process is called **Linear probing**.
- For instance, if the record R3 needs to be entered, the hash function will generate 110 as R3's address. However, the address is already full; as a result, the system selects 113 as the next available data bucket and assigns R3 to it.



❑ Closed Hashing

- When a data bucket is full, a new one is created and connected after the old one, and this method is called **Overflow chaining**.
- For example, if the record R3 needs to be added to the database, the hash function will assign it with the address 110. However, this bucket is too full to accommodate the additional data. In this scenario, a new bucket is placed and linked to the end of 110 buckets.



4. Open and Closed hashing Difference? ***

Difference between Open and Closed Hashing

Closed Hashing:

- ➔ Closed hashing always places keys with same hash function values in same bucket (in overflow buckets also).
- ➔ If bucket is full, the system inserts records in overflow buckets.
- ➔ Different buckets can be of different sizes.
- ➔ Overflow buckets are linked together.

Open Hashing:

- ➔ Open hashing places keys with same hash function values in different bucket if a bucket is full.
- ➔ Set of buckets is fixed; there is no overflow chain.
- ➔ Deletion is difficult in open hashing.

5. Hash Indices structure technique. (Slide-71)- Onno je kono value diyeo korte parbo. Concept thik thaklei hobe. ***

Hash Indices:

- Hashing can be used not only for file organization, but also for index structure creation.
- We construct a hash index as follows. We apply a hash function on a search key to identify a bucket, and store the key and its associated pointers in the bucket.

❑ Hash Indices

